process-scheduler

Description

process-scheduler manages the operation of an arbitrary number of user defined *task*s each of which is made up of a collection of sequentially executed, timed, perhaps repeating, *activities*.

A task is started by a state change on a key value and will continue to operate whilst this trigger value is maintained.

Within a task, activities are executed sequentially and update the Signal K state by modulating a key value as they commences and when they terminates.

Example application

My ship has an electrically powered lubrication pump that delivers grease directly to the propeller shaft upper bearing. I want to ensure that the bearing is well greased at the beginning of every voyage and lightly greased periodically during the voyage.

This requirement is met by a 'shaft-lubrication' task consisting of two activities: a 'start' activity which runs once when the main engine is fired up and a subsequent 'iterate' activity which runs repeatedly for as long as the engine is running. The output of both activities is used to control the operation of the shaft lubrication pump.

The state of my engine ignition switch appears on the Signal K path 'electrical.switches.bank.0.11.state' and my stern gland lubrication pump is operated by a relay on 'electrical.switches.bank.26.5.state'.

The plugin configuration I use for handling the shaft lubrication task looks like this.

```
"configuration": {
  "tasks": [
    {
      "name": "shaft-lubrication",
      "controlpath": "electrical.switches.bank.0.11.state",
      "activities": [
        {
          "name": "start",
          "path": "electrical.switches.bank.26.5.state",
          "delay": 0,
          "duration": 60,
          "iterate": 1
        },
        {
          "name": "iterate",
          "path": "electrical.switches.bank.26.5.state",
          "delay": 1800,
          "duration": 10,
          "iterate": 0
        }
```

```
        ]
      }
    ]
  }
```

Configuration

Scheduler tasks (*tasks*)

Collection of *task* definitions. Each item in the *tasks* array has the following configuration properties.

Task name (*name*)

Required string naming the task being configured.

Task control path (*controlPath*)

Required string supplying a Signal K key and optional trigger value. The general format is '*path*[:*trigger*]'.

When the value on the specified *path* becomes equal to *trigger* (or to 1 if *trigger* is not supplied) then the associated task will start, stopping if and when the trigger value on *path* no longer maintained.

Here are some examples of valid *controlPath*s.

'electrical.switches.bank.0.15.state'. In this case *trigger* is omitted forcing an internal default to 1 making this style useful with Signal K switches and all keys that present a binary value.

'electrical.chargers.276.chargingMode:inverting'. In this case a *trigger* is supplied and when the value on *path* exactly matches *trigger* the associated task will operate.

'notifications.mynotification:alert'. If *path* specifies a key in the 'notifications.' hierarchy then *trigger* must specify a notification state that should operate the associated task.

Activities (*activities*)

Collection of *activity* definitions. Each item in the *activities* array has the following configuration properties.

Activity name (*name*)

Optional string value giving a name to the activity that will be used in notification and logging.

Process control path (*path*)

Required string supplying a Signal K key and optional values that will be used to flag ON and OFF states. The general format is '*path*[:*onState*[:*offState*]]'.

The value on *path* will be updated to *onState* when the associated activity starts. If *offState* is omitted, then *path* will be set to null when the activity stops and otherwise to *offState*. If both *onState* and *offState* are omitted then they will default to 1 and 0 respectively.

Here are some examples of valid *processControlPath*s.

'electrical.switches.bank.12.4.state'. The activity start event will update the key with the value 1; the stop event will update the key with the value 0.

'notifications.mycontrolpath'. The activity start event will issue a notification with state 'normal' (a built in default); the stop event will delete the notification.

'notifications.mycontrolpath:alert' The activity start event will issue a notification with state 'alert'; the stop event will delete the notification.

'notifications.mycontrolpath:alarm:normal' The activity start event will issue a notification with state 'alarm'; the stop event will issue a notification with state 'normal'.

Activity duration in seconds (*duration*)
: Number seconds between activity start and stop events.

Delay start event by this many seconds (*delay*)
: Number of seconds between the activity being triggered and it actually being started.

How many times to repeat (*iterate*)
: Number of times to repeat the activity (0 says forever).

Author

Paul Reeve <*preeve_at_pdjr_dot_eu*>